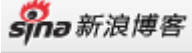




今日热点

资深股民：A股显现三大底部 长期资金迎入场时点

评论 325 点赞 18



只是恬淡寻常一周末

新浪首页 登录 注册

拉登的博客

http://blog.sina.com.cn/ladeng6666 [订阅] [手机订阅]

首页 博文目录 图片 关于我

个人资料



拉登

微博

加好友

发纸条

写留言

加关注



博客等级：**18**
博客积分：**1885**
博客访问：**266,508**
关注人气：**481**
获赠金笔：**2**
赠出金笔：**0**
荣誉徽章：

正文

字体大小：大 中 小

4.1.3 Perlin noise柏林噪声 (2011-12-25 22:07:35)

转 载 ▼

标签： as3.0 bitmap bitmapdata image effects ladeng6666 perlinnoise 分类： 英文翻译教程
it

当我们还是天天想方设法单轮90度骑自行车的小毛孩儿时，电影《Tron》除了让我和我们伙伴有想把飞盘变成导弹的冲动之外，他们还在努力开拓电影中电脑特技的市场，看过这个电影的人都可以证明。这是电脑图形界面的原型(后来被引入到了视频游戏中)。电影中对象的纹理非常平滑，有很好的可塑性，不过人们还是可以从背景中区分出这些对象，因为这些随机的纹理没有晚于融于周围的环境。

Ken Perlin，当时是《Tron》数组应用组的成员之一，被电影中所谓的”机器化”纹理模型搞崩溃了。在完成了这部电影之后，他开发了一种程序化纹理(通过数学算法创建的纹理)，并将这个方法应用到了图像中，来模仿真实的一些纹理，这个方法的效果更加真实，少了数字图像的呆板、僵硬。这些程序化的纹理可以模仿平滑纹理表面，实现烟、火、石头、大理石等大量特效。Perlin的这项研究成果被应用到了现在的3D、视觉特效、图像处理等各种领域中。

Perlin的方法映射到AS3中是BitmapData的perlinNoise()方法(很可惜，按照惯例，perlinNoise()方法的首字母没有大写，但如果写Professor Perlin，是一定要首字母大写的)。这个方法用伪随机像素(一种程序化的随机视觉效果)替换BitmapData中的所有像素。Perlin算法的魅力在于可以分层添加噪点。每个层叫做一个octave，类似于音阶中的八音度。每个octave是下一层octave噪点频率的两倍。这些octave层最后合并到一起，创建随机程度更高的纹理效果。

有句老话是这么说的，一张图片值得用上千条数学公式来解释。图4-4使用BitmapData的perlinNoise()方法创建了几个效果



相关博文

大旺日：12星座8月哪几天运势会起
科技紫微星座网

这条小巷承载了无数成都人饥饿的
更成都

美国博物馆收藏的两件元代玉器欣
古玉居

清华北大直接落户上海|误把学历当
丁金坤

女生恋爱前后颜值变化表
麦拉风

为什么要提倡母乳喂养？真相在这
鲍秀兰

中东部高温闷热持续云南广西等地
中国天气

[转载]1930年代，美军士兵拍摄的
庐西酒徒

纳米比亚的原始部落辛巴人（图）
湖光秋水

西伯利亚呆萌的狐狸
野驴队

更多>>

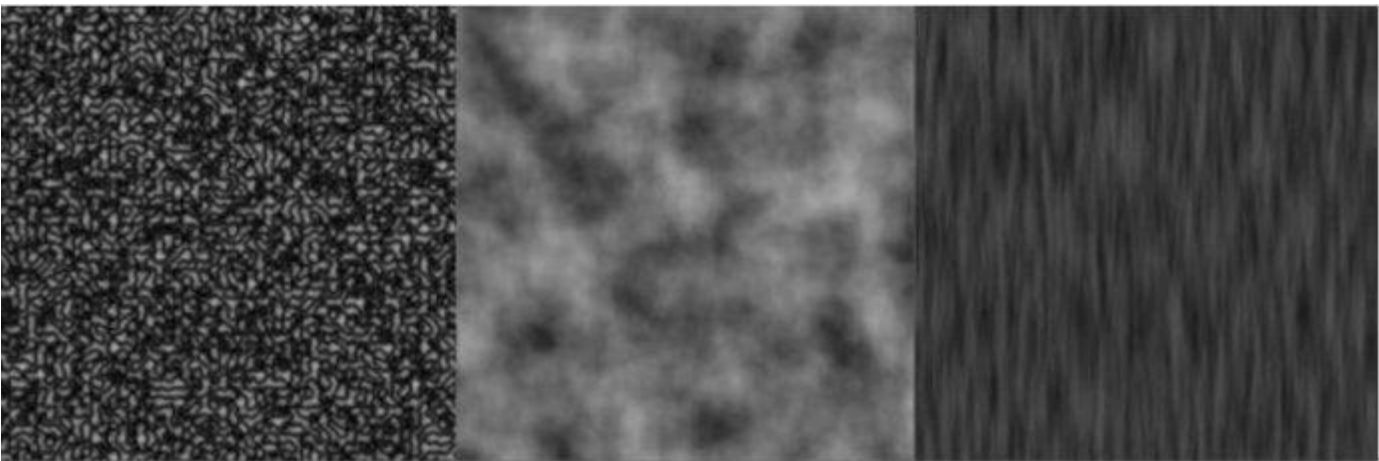


图4-4. perlinNoise() 方法创建的不同纹理效果

这些图像是这么创建的呢？先看下perlinNoise() 方法的结构吧：

```
public function perlinNoise (
    baseX:Number,
    baseY:Number,
    numOctaves:uint,
    randomSeed:int,
    stitch:Boolean,
    fractalNoise:Boolean,
    channelOptions:uint = 7,
    grayScale:Boolean = false,
    offsets:Array = null
) : void
```

它包括很多参数，我们按个看看他们的功能：

- baseX: x轴方向的噪点频率。值越小(噪点频率越高)，噪点细节越丰富，通常这个频率的最大值为所有像素的总数。所以对于一个256px宽的图像，可以设置频率为256。要增加更多的细节，可以将频率降低到128，随着数值越来越小，噪点频率也越来越高，图像程序出的一些细小的斑点。
- baseY: y轴方向的噪点频率。图4-5是一张512px*512px的图像，在baseX和baseY分别为512、128和64时的效果



图4-5. 自做至右baseX和baseY分别为512、128和64的Perlin噪点

- numOctaves: 指定噪点octave层的数量，如果只有一个octave层，那么它就只有一个噪点层。这个噪点层是基于baseX和baseY噪点频率的。每增加一个octave，会相应的增加一个噪点层。这个噪点层的噪点频率是上一层的两倍。例如，设置baseX和baseY分别为256，numOctaves为3，那么第2个噪点层的baseX和baseY分别是128(一个更高的频率)。第3个噪点层的baseX和baseY分别是64。numOctaves值越高，噪点细节越丰富。不过也会消耗更多的CPU，降低动画效果。

图4-6. 一个512px*512px的图像，baseX和baseY为256，numOctaves分别为1、3、6时的效果如下：



推荐博文

- 深夜没在十分钟内回工作微信就辞
- 美团155亿收购摩拜到底值不值
- 网盘衰落之后，为何这款私人云盘
- 短期调整或将仍有反复
- 人工智能与隐私保护
- 台湾科技挣扎，人祸大于天灾？
- 收入份额=市场份额，虎嗅想干什
- 传奇的谢幕，谈岩田聪和他的任天
- 家常主食轻松做之——培根香葱花
- 盘点2015最惊艳流行的婚礼蛋

[查看更多>>](#)



图4-6. baseX和baseY为256，numOctaves分别为1、3、6时将perlinNoise应用到512px*512px的图像上的效果

- randomSeed: 伪随机运算的随机种子，和noise()及pixelDissolve()方法中的randomSeed作用一样
- stitch: 对较大的图像添加噪点时，是否对边缘进行平滑处理，平铺边缘的噪点，实现无缝的纹理
- fractalNoise: 这个参数表示噪点是不规则碎片型(true)，还是流体型(false)，选择流体型噪点后，会产生对比度明显的渐变隔断。形成一种边缘效果。这种效果非常适合模仿海浪、火焰等效果。图4-7分别展示了fractalNoise分别为true和false的效果



图4-7. 左图为不规则噪点fractalNoise为true，右图为流体型噪点fractalNoise为false

- channelOptions: 决定噪点所在的通道，和noise()方法中的channelOptions作用相同
- grayScale: 表示噪点是灰阶还是在channelOptions参数中定义的通道中添加噪点
- offsets: 这是一个包含一组坐标的数组。每个坐标影响一个octave层。如数组中的第一个坐标影响第一个octave层，第二个坐标影响第二个octave层，以此类推。随时间逐渐修改这些坐标值，可以创建动态的噪点效果，这也是一些烟火效果的基本原理。注意，因为不同的店控制不同的octave层，所以可以为每个octave层设置不同的坐标和方向

perlinNoise() 包含了很多的参数，这些参数可以帮助我们实现大量的特效，如海浪、烟火，或者木材、锈迹大理石纹理等。我将在本书的后续章节中，与你一同学习这些效果的制作方法。不过现在，我们还是先通过一个简单的实例，了解一下这些参数的实际效果吧！

打开本章源文件中的PerlinNoiseTest.as可以得到完整的代码如下：

```
private function onSpriteEnterFrame(event:Event):void {
    var point:Point;
    var direction:int;
    for (var i:uint = 0; i < NUM_OCTAVES; i++) {
        point = _offsets[i] as Point;
        // even loop iterations will move in a negative direction, odd loops
```

```
will be positive

        direction = (i%2==0) ? -1 : 1;
        // higher octaves will move more quickly than lower octaves
        point.x += OFFSET_X_RATE/(NUM_OCTAVES-i)*direction;
        point.y += OFFSET_Y_RATE/(NUM_OCTAVES-i)*direction;
    }
    makePerlinNoise();
}

}
```

测试文件，默认得到的动画效果如图4-8：



图4-8. perlinNoise() 中以不同的速度场两个不同的方向移动的octave形成的动画效果

在动画效果中，两个不同的octave层以不同的的速度朝不同的方向移动。仔细看一下类文件，可以发现在构造函数中创建了一个BitmapData对象，作为噪点的应用对象。然后创建了一个长度为NUM_OCTAVES的数组_offsets。最后调用makePerlinNoise()方法，创建噪点，并将BitmapData应用到一个新的Bitmap对象中，添加到舞台上。然后添加一个ENTER_FRAME事件侦听。

makeNoise()方法仅仅只是调用了perlinNoise()方法，并使用了代码顶部定的_offsets数组。这个数组会再onSpriteEnterFrame事件处理函数中实时更新。在for循环中的代码首先更具索引值的奇偶性来定义噪点移动的方向。然后依据这个方向修改数组中的Point。

```
direction = (i%2==0)? -1:1;
```

如果i%2没有余数，则i为偶数direction取-1。若为奇数，则direction取1。

Octave的移动速率与octave层级有关，最高层的octave的移动速率分别是OFFSET_X_RATE和OFFSET_Y_RATE。因为当for循环进行到最后一次迭代时，i等于(NUM_OCTAVES-1)，那么NUM_OCTAVES-(NUM_OCTAVES-1)就等于1，然后用它作为分母计算偏移量。octave级数越低，偏移量就越小，octave移动的也就越慢。

举例说明，假设octave层的移动速度OFFSET_X_RATE为2，OFFSET_Y_RATE为50。在for循环的第一次迭代中(i=0)，direction为-1，此时，point.x和point.y的计算公式如下：

```
point.x += 2 / (3 - 0) * -1; //约为 -.667
point.y += 5 / (3 - 0) * -1; //约为 1.667
```

当i=1时，direction为1，测试point.x和point.y的计算公式如下：

```
point.x += 2 / (3 - 1) * 1; //约为 1
point.y += 5 / (3 - 1) * 1; //约为 2.5
```

在最后一次迭代时i=2，direction还是-1，point.x和point.y分别为：

```
point.x += 2 / (3 - 0) * -1; //约为 -.667
point.y += 5 / (3 - 0) * -1; //约为 1.667
```

效果是层级高的octave比层级低的运动的要快。
源码顶部将perlinNoise()的所有参数都保存成常量，这样我们可以轻松修改这些参数。接下来我们看看下面这些参数的效果。
参数修改后如图4-9.



图4-9. octave的数量改为1，baseX和baseY改为500后的效果。
如果希望有更多的噪点细节，可以降低BASE_X和BASE_Y提高噪点频率。试着值降低其中一个值，看看效果如何。

```
private static const BASE_X:Number = 100;
```

图4-10. 增加x轴的噪点频率之后的效果.



图4-10. baseX降至100后，生成一种垂直模式.
频率增加的在高些，此时的噪点效果更像纺织和木头纹理了，如图4-11.

```
private static const BASE_X:Number = 5;  
private static const BASE_y:Number = 200;
```

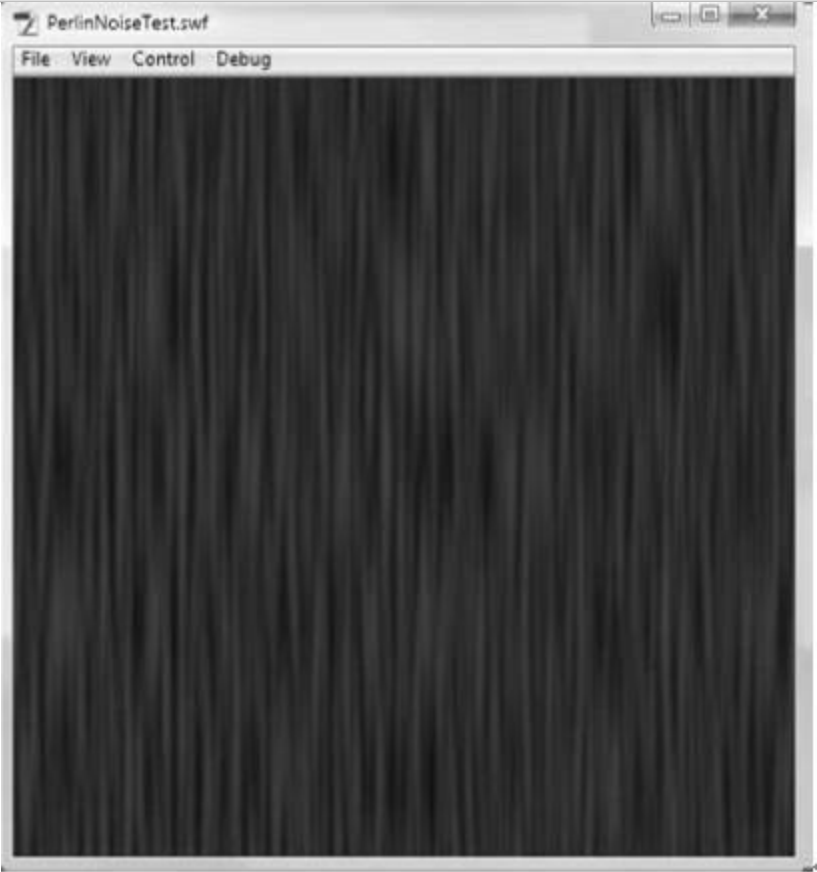


图4-11. baseX将至5，baseY降至100后产生的类似纺织的效果
接下来调整octave的层数，看看效果如何。首先降低BASE_X和BASE_Y，分别设置为200，octave层数设置为1，效果如图4-12。

```
private static const BASE_X:Number = 200;
private static const BASE_y:Number = 200;
```



图4-12. baseX和baseY分别为200，octave层数为1时的噪点效果
然后敬爱那个NUM_OCTAVES改成3，添加更多的噪点细节，如图4-13。

```
private static const NUM_OCTAVES:Number = 3;
```

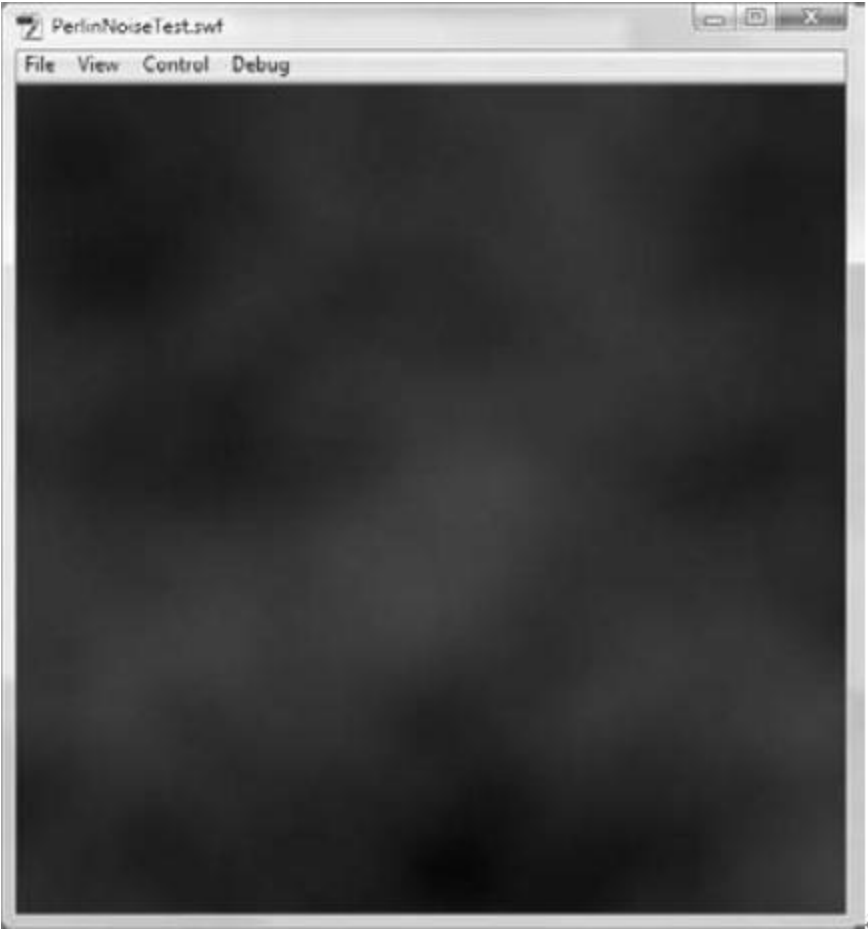


图4-13. octaves层数升为3之后的效果
再次声明，我们现在做的是布置3个不同的噪点图像，每个图像的噪点频率是前一个噪点两倍。这些噪点图的模式都默认为无规则碎片模式(fractal)，将其修改为流体模式(turbulence)，得到另外一个同的效果，如图4-14：

```
private static const FRACTAL:Boolean = false;
```

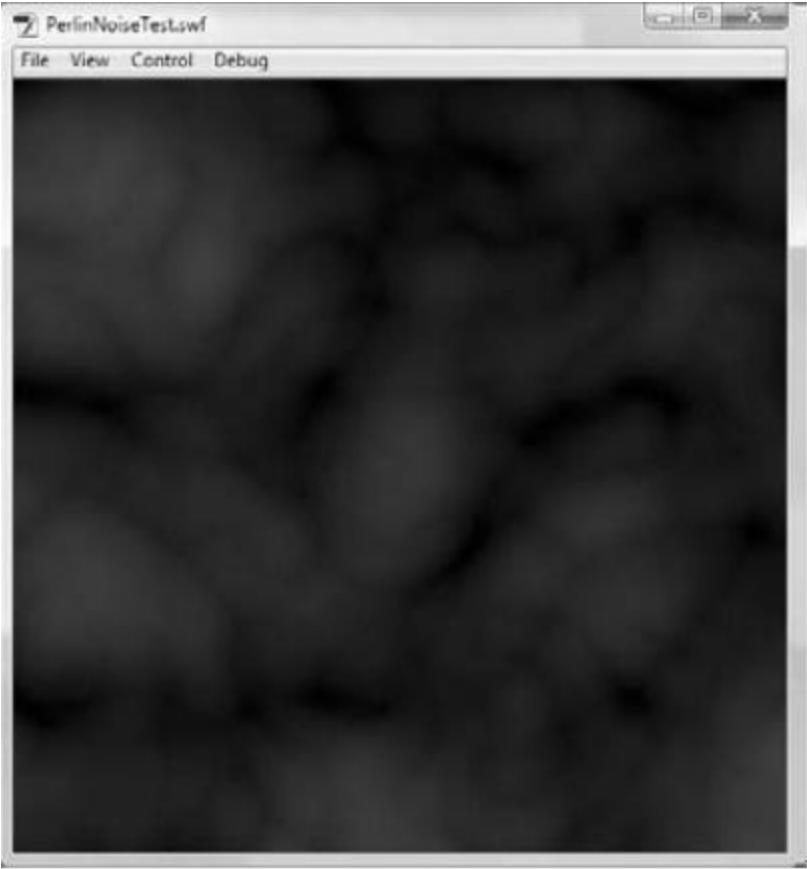



图4-14. fractalNoise参数修改为false后，得到的流体效果

最后，我们用两个通道来创建一个多彩的噪点图像。记住，这跟noise()中的通道参数用法是一样的。你可以使用BitmapDataChannels中的任意一个常量，或者通过位运算将多个通道合并后使用。如图4-15，因为红色和蓝色通道合并，所以我们可以看到紫色的噪点：

```
private static const CHANNELS:uint = BitmapDataChannel.RED |  
BitmapDataChannel.BLUE;
```

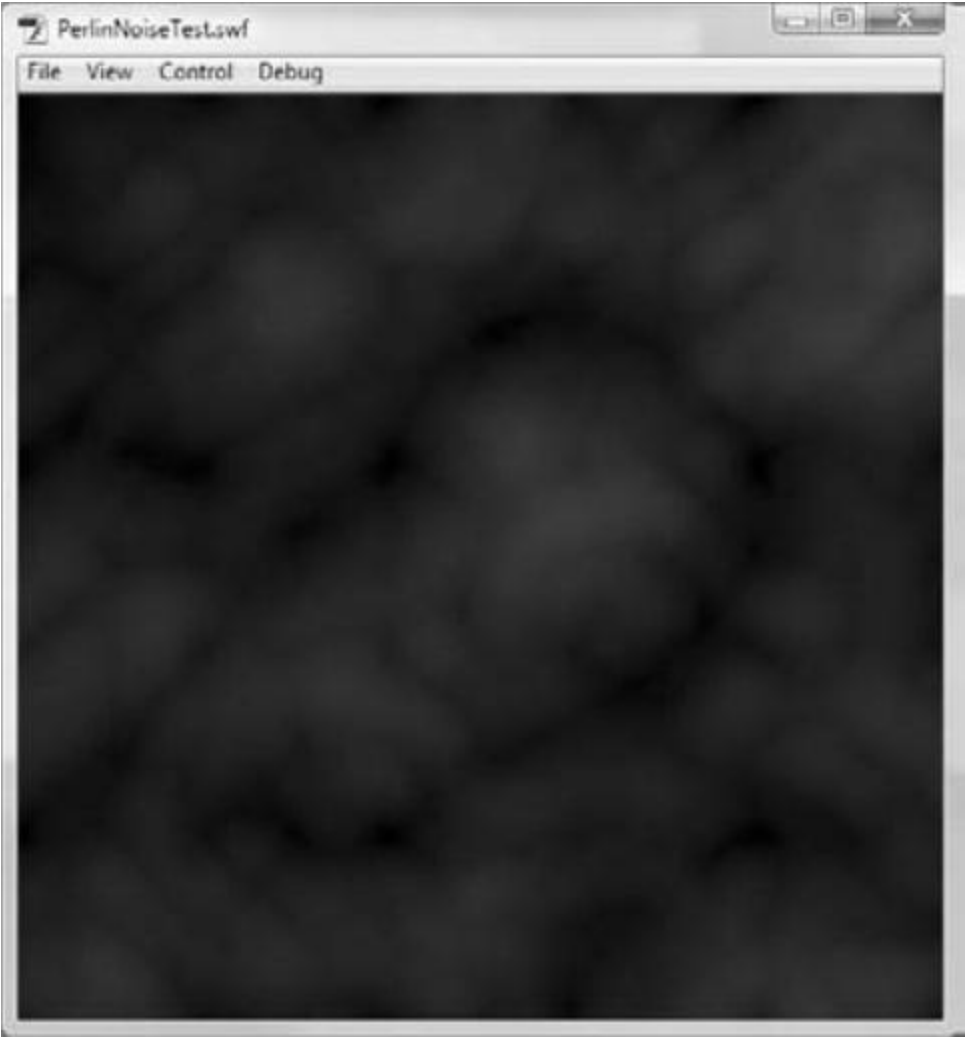


图4-15. 使用红色和蓝色通道后的噪点效果(截图为黑白图像，效果不是很明显)
通过修改不同的参数，我们还可以实现很多同的组合效果，比如offset值，试着修改这些参数，看看他们能产生什么效果。在本书的后续章节中，很多实例都用到了perlinNoise()方法，所以扎实的掌握perlinNoise()的用法是非常重要的。

1

喜欢

0

赠金笔

分享:

阅读(1931) | 收藏(0) | 转载(1) | 喜欢▼ | 打印 | 举报

已投稿到: 排行榜

前一篇: [HTML5基础入门]——第七章 迷宫游戏

后一篇: [HTML5基础入门]——第8章. 剪刀石头布

新浪BLOG意见反馈留言板 电话：4006900000 提示音后按1键（按当地市话标准计费） 欢迎批评指正
新浪简介 | About Sina | 广告服务 | 联系我们 | 招聘信息 | 网站律师 | SINA English | 会员注册 | 产品答疑

Copyright © 1996 - 2018 SINA Corporation, All Rights Reserved
新浪公司 版权所有